30-Day Android Challenge

Senior Android interview prep — Junior → Mid → Senior → Architect

2026 Edition

DAY 26

Advanced Kotlin — Generics, Inline & Reified, DSLs, Context Parameters, K2 & KSP



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 26

Why advanced Kotlin is Day 26

Senior interviews don't ask "what is a data class". They ask **why** the language behaves the way it does — what happens at compile time, what's left at runtime, and what it costs:

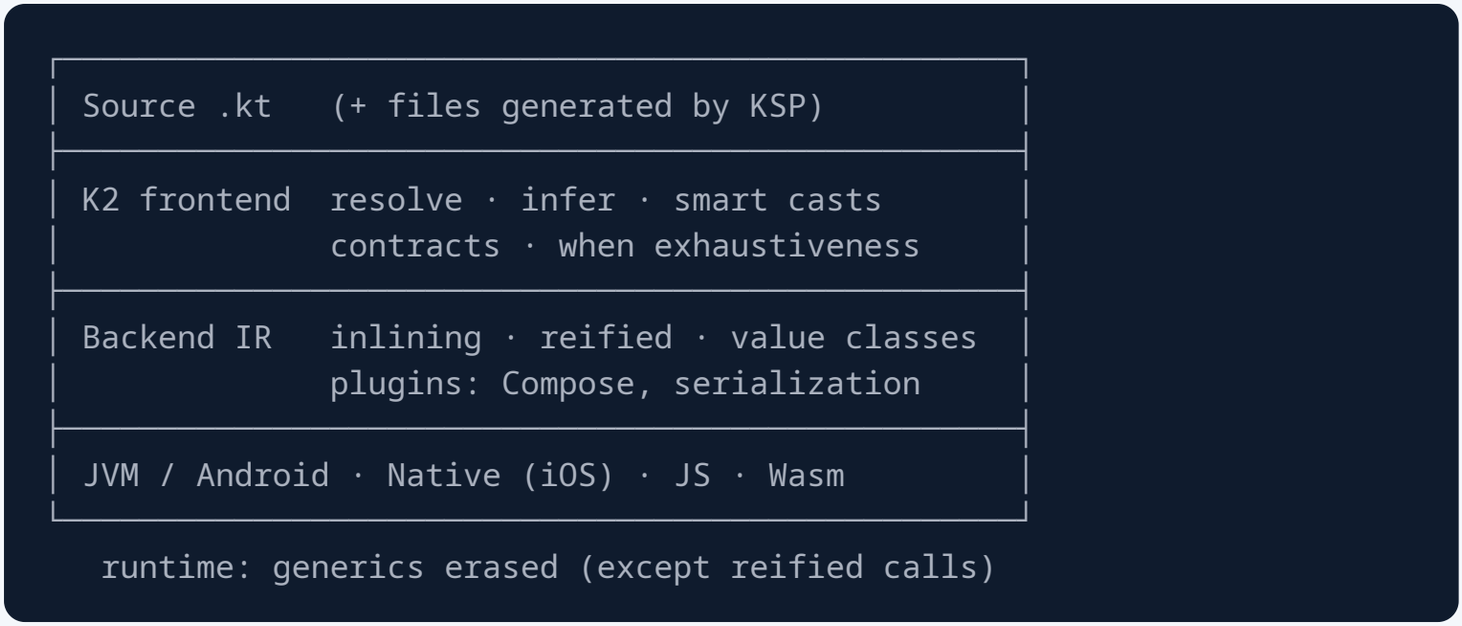
- `inline`, `reified` and type erasure — the classic "explain the bytecode" filter
- Sealed and value classes for type-safe domain models
- Variance: why `List<String>` is a `List<Any>` but `MutableList<String>` isn't
- K2 is the default compiler since Kotlin 2.0; kapt is in maintenance mode, KSP is the standard
- KMP puts `expect` / `actual` and public API design on the table

Level	Questions
Junior	Inline functions · Reified & erasure · Sealed hierarchies
Mid	Generics & variance · Value classes · Delegation · DSLs
Senior	Context parameters · Contracts & smart casts · The K2 compiler
Architect	KSP & metaprogramming · expect/actual · Public API design



Esmaeeil Moradi
@esmaeeil-moradi700

Mental model: compile time vs runtime



- Most "advanced" features are **compile-time**: they cost nothing at runtime — until something forces boxing or allocation
- On the JVM, generic type arguments are **erased**; only inline + `reified` brings them back, per call site
- The compiler **trusts** some declarations (contracts, `@UnsafeVariance`, Compose's `@Stable`) — a wrong promise is your bug

THE QUESTION BEHIND EVERY ANSWER TODAY

"Does this exist at runtime, or only in the compiler?" — Inline, reified, value classes, contracts and KSP all make sense once you answer it.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 26



Junior

Inline functions · Reified & erasure · Sealed hierarchies



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. Inline functions: what really gets inlined

```
inline fun <T> measure(tag: String, block: () -> T): T {
    val start = System.nanoTime()
    try {
        return block()
    } finally {
        Log.d(tag, "took ${System.nanoTime() - start} ns")
    }
}

fun findUser(users: List<User>, id: String): User? {
    users.forEach { user ->
        if (user.id == id) return user // non-local return
    }
    return null
}
```

- The compiler copies the function body **and the lambda** into the call site: no function object, no extra call
- Lambdas passed to inline functions may use a **non-local `return`** — `forEach` is inline, so `return user` leaves `findUser`
- Inlining is also what makes `reified` possible (Q2)



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. noinline and crossinline

Parameter	Meaning	When
(default)	Lambda is inlined	No allocation; non-local return allowed
`noinline`	Lambda stays a **real object**	You store it or pass it to a non-inline function
`crossinline`	Inlined, but non-local `return` forbidden	The lambda runs inside another lambda or object

```
inline fun View.onClick(crossinline action: () -> Unit) {
    setOnClickListener { action() } // runs in another lambda
}
```

INTERVIEW TRAP

"*`noinline` avoids creating a lambda object.*" — Backwards, and v1 said it. `noinline` **creates** the object, because you need a value you can store or pass on.

INTERVIEW TRAP

"Inline everything for speed." — Without lambda parameters the gain is negligible (the IDE warns), and every call site grows the bytecode. A public inline body is also copied into callers — changing it requires them to recompile.



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. Type erasure and reified

```
// Doesn't compile: "Cannot check for instance of erased type"
fun <T> List<Any>.firstOfType(): T? =
    firstOrNull { it is T } as T?

inline fun <reified T> List<Any>.firstOfType(): T? =
    firstOrNull { it is T } as T?

inline fun <reified T : Activity> Context.open(
    vararg extras: Pair<String, Any?>,
) {
    startActivity(
        Intent(this, T::class.java).putExtras(bundleOf(*extras))
    )
}

context.open<DetailActivity>("id" to 42L)
```

- On the JVM, `List<String>` and `List<Int>` are the **same class** at runtime
- `reified` works only in **inline** functions: the compiler substitutes the real type at every call site, so `is T`, `T::class` and `typeOf<T>()` work



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. What interviewers check

INTERVIEW TRAP

"`reified` *keeps generic types at runtime.*" — Only inside that inline call. Classes stay erased; a reified function can't be called from Java, and `T` must be known at the call site.

INTERVIEW TRAP

"`T::class` *is enough for* `List<User>`." — It gives `List`, not `List<User>`. For nested type arguments use `typeOf<T>()` — this is how `Json.decodeFromString<List<User>>(json)` finds the right serializer.

FOLLOW-UP

"Where do you use reified every day?" — `filterIsInstance<T>()`, `viewModel<T>()`, Koin's `get<T>()`, kotlinx.serialization's `decodeFromString<T>()`, Ktor's `body<T>()`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. Sealed hierarchies and exhaustive when

```
sealed interface PaymentResult {
    data class Success(val receiptId: String) : PaymentResult
    data class Declined(val reason: String) : PaymentResult
    data object Cancelled : PaymentResult
}

fun message(r: PaymentResult): String = when (r) {
    is PaymentResult.Success -> "Paid: ${r.receiptId}"
    is PaymentResult.Declined if r.reason == "3DS" ->
        "Please confirm in your bank app"
    is PaymentResult.Declined -> "Declined: ${r.reason}"
    PaymentResult.Cancelled -> "Cancelled"
}
```

	enum	sealed class	sealed interface
Instances	Fixed singletons	Any number, own state	Any number, own state
Subtypes carry	The same properties	Different data	Different data
Shared constructor / state	Yes	Yes	No
A class can have several	—	No (one superclass)	Yes



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. Sealed-class traps

INTERVIEW TRAP

"The compiler warns when a `when` isn't exhaustive." — v1 said it. Since Kotlin 1.7 a non-exhaustive `when` over a sealed type, enum or `Boolean` is a compile **error**, for statements as well as expressions.

INTERVIEW TRAP

Adding `else ->` to a `when` over a sealed type. It switches exhaustiveness off: the next subtype someone adds silently falls into `else` instead of failing the build.

- Subclasses of a sealed type must live in the **same module and package**
- **Guard conditions** (`is X if cond ->`, previous slide) are stable since Kotlin 2.2

FOLLOW-UP

"Sealed class or sealed interface?" — Interface by default: more flexible, a type can implement several. Sealed class when subtypes share constructor state.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 26



Mid-Level

Generics & variance · Value classes · Delegation · DSLs



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Variance: in, out and invariant

```kotlin
interface Source<out T> {       // producer: only returns T
    fun next(): T
}

interface Sink<in T> {          // consumer: only accepts T
    fun put(item: T)
}

fun demo(strings: Source<String>, anySink: Sink<Any>) {
    val a: Source<Any> = strings    // OK: out = covariant
    val b: Sink<String> = anySink   // OK: in = contravariant
}
```

Kotlin	Java	Meaning
`out T`	`? extends T`	Producer — you can read `T`
`in T`	`? super T`	Consumer — you can pass `T` in
`*`	`?`	Unknown type — read as `Any?`, can't write
`T`	`T`	Invariant — read and write



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Use-site variance and bounds

```
// Use-site variance: this call only reads `from`
fun copy(from: Array<out Number>, to: Array<in Number>) {
    for (i in from.indices) to[i] = from[i]
}

// Several upper bounds
fun <T> sortAll(items: MutableList<T>)
    where T : Named, T : Comparable<T> = items.sort()

// Definitely non-nullable: T & Any
fun <T> T.orThrow(): T & Any = this ?: error("null")
```

INTERVIEW TRAP

"*`MutableList<String>` is a `MutableList<Any>`*." — **No:** it's invariant, otherwise you could add an `Int` into a list of strings. `List<String>` **is** a `List<Any>`, because it's declared `List<out E>` and is read-only.

FOLLOW-UP

"Then how does `List<out E>` have `contains(element: E)`?" — The stdlib marks it `@UnsafeVariance`: `contains` only reads, so the unsafe position is deliberately allowed.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Value classes: type safety without a wrapper object

```
@JvmInline
value class UserId(val raw: String) {
    init { require(raw.isNotBlank()) { "blank id" } }
}

@JvmInline
value class Cents(val value: Long) {
    operator fun plus(other: Cents) = Cents(value + other.value)
}

fun load(id: UserId) { /* an OrderId can't be passed here */ }
```

- Compiled to the underlying value (`String`, `long`) wherever it's used as its own static type
- **Boxed** when used as a generic type argument (`List<UserId>`), as a nullable of a primitive (`Cents?`), or as `Any` / an interface
- Functions that take value classes get **mangled** JVM names — awkward to call from Java

INTERVIEW TRAP

"Inline classes have zero overhead." — v1 said it. Only when they aren't boxed (see above). And `inline class` is the old, deprecated syntax: use `@JvmInline value class`. Identity (`===`) isn't allowed; equality is by value.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Property delegation

```
class SavedDelegate<T>(
    private val handle: SavedStateHandle,
    private val default: T,
) : ReadWriteProperty<Any?, T> {
    override fun getValue(
        thisRef: Any?, property: KProperty<*>,
    ): T = handle.get<T>(property.name) ?: default

    override fun setValue(
        thisRef: Any?, property: KProperty<*>, value: T,
    ) {
        handle[property.name] = value
    }
}

class SearchViewModel(handle: SavedStateHandle) : ViewModel() {
    var query: String by SavedDelegate(handle, "")
}
```

- `by` routes a property's getter and setter to another object: `getValue` / `setValue` operators
- Built in: `lazy`, `Delegates.observable` / `vetoable`, `by map`, delegating to another property (`by ::other`)
- Android uses it everywhere: `by viewModels()`, `by remember { mutableStateOf(…) }`



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Class delegation — and its traps

```
class LoggingList<T>(
    private val inner: MutableList<T> = mutableListOf(),
) : MutableList<T> by inner {
    override fun add(element: T): Boolean {
        println("add $element")
        return inner.add(element)
    }
}
```

INTERVIEW TRAP

"*`addAll()` goes through my overridden `add()`*." — **No.** Delegation isn't inheritance: `addAll` is forwarded to `inner` and handled **there**. Your override never sees those elements.

INTERVIEW TRAP

`lazy { }` is `SYNCHRONIZED` by default — thread-safe, with a lock. Use `LazyThreadSafetyMode.PUBLICATION` or `NONE` only when you know the threading, never "for speed" by default.



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Type-safe builders (DSLs)

```kotlin
@DslMarker annotation class FormDsl

@FormDsl
class FormBuilder {
    private val fields = mutableListOf<Field>()
    fun text(name: String, init: FieldBuilder.() -> Unit = {}) {
        fields += FieldBuilder(name).apply(init).build()
    }
    fun build() = Form(fields.toList())
}

@FormDsl
class FieldBuilder(private val name: String) {
    var required = false
    var maxLength = Int.MAX_VALUE
    fun build() = Field(name, required, maxLength)
}

fun form(block: FormBuilder.() -> Unit): Form =
    FormBuilder().apply(block).build()
```

- The core is the **lambda with receiver** `FormBuilder.() -> Unit`: inside, `this` is the builder
- Add extension functions, `infix` and `operator invoke` when they make the call site read naturally



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Using the DSL — and what @DslMarker really does

```
val signUp = form {
    text("email") { required = true }
    text("bio") {
        maxLength = 160
        text("nested")          // error: implicit outer receiver
        this@form.text("x")     // compiles: explicit receiver
    }
}
```

INTERVIEW TRAP

"`@DslMarker` *makes wrong nesting impossible."* — It only blocks **implicit** access to outer receivers. An explicit `this@form` still compiles — v1 overstated it.

INTERVIEW TRAP

A DSL that does work while it's being built (network calls, writes). Build an **immutable model** first, then interpret or validate it — testable and easy to reason about.

FOLLOW-UP

"Is Compose a builder DSL?" — No. Composables are functions transformed by the Compose compiler plugin. Its scopes (`RowScope` , `ColumnScope`) do use a `@DslMarker` — `@LayoutScopeMarker` .



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 26



Senior

Context parameters · Contracts & smart casts · The K2 compiler



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Context parameters

KOTLIN 2.2 (PREVIEW)

Context parameters arrived as a **preview** in Kotlin 2.2, enabled with the `-Xcontext-parameters` compiler flag. They replace the experimental **context receivers**, which are deprecated. Check your Kotlin version's release notes before using them in a public API.

```
interface Logger { fun info(msg: String) }
interface Clock { fun now(): Long }

context(logger: Logger, clock: Clock)
fun startSync(job: SyncJob) {
    logger.info("Sync ${job.id} at ${clock.now()}")
}

fun runSync(job: SyncJob, logger: Logger, clock: Clock) {
    with(logger) {
        with(clock) {
            startSync(job) // both resolved from context
        }
    }
}
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Receivers vs parameters — and when to use them

	Context receivers (deprecated)	Context parameters
Declared as	`context(Logger)`	`context(logger: Logger)`
Members inside	Implicitly available: `info(…)`	Accessed by name: `logger.info(…)`; `_` if unnamed
Status	Experimental, deprecated	Preview since Kotlin 2.2

- Good fit: **cross-cutting capabilities** passed through many calls — logging, a transaction, a typed-error scope
- Not a replacement for constructor injection of services (Day 7)

INTERVIEW TRAP

"Context parameters are just context receivers renamed." — They're a redesign: a context parameter is **not** an implicit `this`, so members don't leak into the function body.

INTERVIEW TRAP

Threading a dozen capabilities through context parameters "to avoid DI". Every caller must now provide them — hidden coupling that's harder to follow than a constructor.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Contracts: teaching the compiler

```
@OptIn(ExperimentalContracts::class)
fun Order?.isPayable(): Boolean {
    contract { returns(true) implies (this@isPayable != null) }
    return this != null && status == Status.OPEN
}

fun pay(order: Order?) {
    if (order.isPayable()) {
        submit(order.id) // smart cast: Order, not Order?
    }
}
```

- `returns(…) implies (…)`: a smart cast after the call — like the stdlib's `isNullOrEmpty()` and `require()`
- `callsInPlace(block, EXACTLY_ONCE)`: the lambda runs here, once — so a `val` can be initialized inside it, as with `run { }`
- Still behind the `@OptIn(ExperimentalContracts::class)` opt-in

INTERVIEW TRAP

"The compiler checks my contract." — It **doesn't**. It trusts it. A wrong contract gives you a smart cast to non-null on a value that can be null — a crash the type system promised couldn't happen.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. The K2 compiler: what changed

- **Kotlin 2.0** made K2 stable and the default: a new frontend (FIR) and one pipeline for JVM, Native, JS and Wasm
- **Faster compilation** in JetBrains' benchmarks — measure your own build before and after
- The **Compose compiler** moved into the Kotlin repository: plugin `org.jetbrains.kotlin.plugin.compose`, same version as Kotlin
- **Smarter smart casts:**

```
// interface Ok : Status, interface Postponed : Status
fun handle(s: Any) {
    if (s is Ok || s is Postponed) {
        s.signal() // K2: smart cast to the common Status
    }
}

fun pet(animal: Any) {
    val isCat = animal is Cat
    if (isCat) animal.purr() // K2: smart cast via local val
}
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Migrating a large codebase to K2

Area	What to do
Annotation processors	Move from kapt to KSP (Room, Moshi, Hilt/Dagger support it)
Compiler plugins	K2-compatible versions; Compose, serialization and Parcelize ship with Kotlin
Language changes	Read the K2 compatibility guide; fix warnings **before** upgrading
Static analysis	detekt, ktlint and Lint versions that understand K2
IDE	Enable K2 mode in Android Studio / IntelliJ so the IDE matches the compiler

INTERVIEW TRAP

"K2 is only a faster compiler." — It's a rewritten frontend. A few edge cases in resolution and smart casts behave differently, and every compiler plugin had to be ported.

INTERVIEW TRAP

"The K2 plugin API is stable — write a compiler plugin for codegen." — v1 implied it. The compiler plugin API is **not** stable and breaks between Kotlin versions. Generating code? Use KSP (Q11).



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 26

Architect

KSP & metaprogramming · expect/actual · Public API design



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Choosing a metaprogramming tool

Tool	Runs	Can	Cost / risk
Reflection (`kotlin-reflect`)	Runtime	Inspect anything	Slow first access, big dependency, R8 keep rules
kapt	Compile time, via Java stubs	Run Java annotation processors	Slow stub generation; maintenance mode
KSP	Compile time, Kotlin-aware	Read declarations, generate **new** files	Can't modify code or see function bodies
Compiler plugin	Inside the compiler	Transform code (Compose, serialization)	Unstable API, breaks between Kotlin versions
Lint / detekt rule	Static analysis	Report and auto-fix patterns	No code generation

INTERVIEW TRAP

"Dagger and kotlinx.serialization use reflection." — v1 said it. Dagger generates code at compile time; kotlinx.serialization uses a **compiler plugin**. That's why they're fast and R8-friendly.



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. A KSP processor

```
class RouteProcessor(
    private val codeGenerator: CodeGenerator,
    private val logger: KSPLogger,
) : SymbolProcessor {
    override fun process(
        resolver: Resolver,
    ): List<KSAnnotated> {
        val (valid, deferred) = resolver
            .getSymbolsWithAnnotation("com.app.Route")
            .filterIsInstance<KSClassDeclaration>()
            .partition { it.validate() }
        valid.forEach { generateRoute(it) }
        return deferred // retried in the next round
    }
}

class RouteProcessorProvider : SymbolProcessorProvider {
    override fun create(env: SymbolProcessorEnvironment) =
        RouteProcessor(env.codeGenerator, env.logger)
}
```

- Register the provider in `META-INF/services`; generate files with KotlinPoet
- **Incremental:** pass `Dependencies(aggregating = false, file)` so only affected outputs rebuild
- Report errors with `logger.error(msg, symbol)` — they point at the user's source line



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. expect/actual — keep it thin

```
// commonMain
expect fun logDebug(tag: String, message: String)

interface SecureStore {
    suspend fun put(key: String, value: String)
    suspend fun get(key: String): String?
}

// androidMain
actual fun logDebug(tag: String, message: String) {
    android.util.Log.d(tag, message)
}
class KeystoreStore(ctx: Context) : SecureStore { /* … */ }

// iosMain
actual fun logDebug(tag: String, message: String) {
    println("[$tag] $message")
}
class KeychainStore : SecureStore { /* … */ }
```

- `expect fun` for small leaf functions; **interfaces + DI** for services — they're fakeable in common tests
- `actual typealias` maps an expectation onto an existing platform type



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Multiplatform traps

INTERVIEW TRAP

Reaching for `expect class` first. `expect` / `actual` **classes** are still in Beta and produce a warning unless you opt in with `-Xexpect-actual-classes`. An interface implemented per platform is simpler and testable.

INTERVIEW TRAP

Writing `expect` / `actual` for what common libraries already provide: dates (`kotlinx-datetime`), I/O (`kotlinx-io`), HTTP (Ktor), serialization. Check the ecosystem first.

- The **default hierarchy template** gives you shared `iosMain`, `appleMain` and `nativeMain` source sets without manual wiring
- Most shared code is plain `commonMain` Kotlin; `expect` / `actual` is the thin edge — see Day 3 for sharing ViewModels



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Designing a public Kotlin API

```
// build.gradle.kts
kotlin { explicitApi() }
```

```
@RequiresOptIn(message = "May change without notice")
public annotation class ExperimentalPaymentsApi

public class PaymentClient internal constructor(
    private val engine: Engine,
) {
    @ExperimentalPaymentsApi
    public suspend fun walletPay(r: WalletRequest): Receipt =
        engine.wallet(r)

    @Deprecated(
        "Use pay(PaymentRequest)",
        ReplaceWith("pay(PaymentRequest(amount))"),
        DeprecationLevel.ERROR,
    )
    public suspend fun pay(amount: Long): Receipt =
        pay(PaymentRequest(amount))
}
```

- **Explicit API mode:** every public declaration needs a visibility modifier and an explicit type
- `@RequiresOptIn` for unstable parts; deprecate in steps `WARNING` → `ERROR` → `HIDDEN`, with `ReplaceWith`



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Binary compatibility traps

```kotlin
public inline fun <reified T> Json.decodeOrNull(s: String): T? =
    decodeCatching { decodeFromString<T>(s) }

@PublishedApi
internal inline fun <T> decodeCatching(block: () -> T): T? =
    try { block() } catch (e: SerializationException) { null }
```

INTERVIEW TRAP

A **data class** in a public API. Adding a property changes the signatures of the constructor and `copy()` — a binary-incompatible change for every client compiled against the old version. Prefer a regular class, or a builder.

INTERVIEW TRAP

A public inline function calling `internal` code. Inline bodies are copied into clients, so the compiler requires `@PublishedApi` — and that internal function is now part of your ABI forever.

SENIOR ANSWER IN ONE LINE

"Know what the compiler does for you and what's left at runtime: inline and reified are call-site copies, value classes box in generic positions, contracts are trusted promises — and KSP generates code, it never rewrites it."



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 26 · ARCHITECT

Day 26 — cheat sheet

- `inline` copies body + lambdas into the call site; `noinline` **keeps** the object; `crossinline` forbids non-local return.
- Generics are erased at runtime; `reified` works only in inline functions — use `typeOf<T>()` for nested types.
- Non-exhaustive `when` on sealed types is an **error** (Kotlin 1.7+); never add `else` to a sealed `when`.
- `out` = producer, `in` = consumer; `List<out E>` is covariant, `MutableList<E>` is invariant.
- `@JvmInline value class` boxes as a generic argument, as a nullable primitive, or as `Any`.
- Class delegation forwards calls — the delegate's internal calls never reach your overrides.
- `@DslMarker` blocks only **implicit** outer receivers; build an immutable model, then interpret it.
- Context parameters: preview since Kotlin 2.2, replacing deprecated context receivers. Contracts are trusted, not checked.
- K2 is the default since Kotlin 2.0: kapt → KSP; compiler plugins aren't a stable API.
- KMP: thin `expect` / `actual`, interfaces + DI for services. Public APIs: explicit API mode, no data classes, `@PublishedApi` with care.

Day 27 is next → Advanced Architecture Patterns. Save this post and follow along.



Esmaeeil Moradi
@esmaeeil-moradi700